

WHY 90% OF JAVA DEVELOPERS **FAIL** TOP-TIER INTERVIEWS



@Java_in.depth

"Hi, I'm Vishal, Founder of Java In Depth and a former Senior Engineer at Microsoft.

I've interviewed hundreds of candidates. Most can write a 'For-Loop', but 70% fail when asked about Thread Pools, Bean Lifecycles, or Database Indexing.

If you want a 25LPA+ package, you need to stop being a 'Coder' and start being an 'Engineer'. This guide is your shortcut to mastering the fundamentals that top-tier companies actually care about.

1. MULTITHREADING DEEP CONCEPTS

- **Thread Pool & ExecutorService:** Managing system resources efficiently.
- **Callable vs Runnable & Future:** Handling return values in async tasks.
- **CompletableFuture:** The modern way to handle asynchronous pipelines.
- **Concurrency Issues:** Deadlock detection, Race Conditions, and Starvation.
- **Advanced Locking:** ReentrantLock vs. synchronized blocks.

2. DESIGN PATTERNS (GOF)

- **Singleton:** Ensuring a single instance (Must-know: Thread-safe implementation).
- **Factory:** Decoupling object creation from business logic.
- **Builder:** Solving complex object construction (Telescoping Constructor).
- **Strategy:** Swapping algorithms at runtime (e.g., Payment Gateways).
- **Observer:** Building reactive event-driven systems.
- **Adapter:** Plugging incompatible interfaces together.
- **Decorator:** Adding functionality without class inheritance.

3. SPRING BOOT INTERNAL WORKING

- **IoC Container & Dependency Injection:** How Spring manages your object graph.
- **Bean Lifecycle:** Understanding @PostConstruct to @PreDestroy.
- **Annotations Internals:** @Autowired, @Component, @Service, vs. @Repository.
- **Auto-Configuration:** How @SpringBootApplication scans and configures beans.
- **DispatcherServlet:** The internal Request-Response journey.

4. MICROSERVICES ARCHITECTURE

- **API Gateway & Service Discovery:** The entry point and registry of your system.
- **Load Balancer & Circuit Breaker:** Ensuring system resilience (Resilience4j).
- **Communication:** Event-Driven Architecture using Kafka or RabbitMQ.
- **Distributed Transactions:** Managing data consistency with the Saga Pattern.
- **Monolith vs Microservices:** Knowing when (and when not) to decouple.

5. DATABASE DEEP CONCEPTS

- **Indexing:** How B-Trees work and optimizing SQL queries.
- **ACID & Transactions:** Ensuring reliability under heavy load.
- **Isolation Levels:** Managing concurrent data access (Read Committed vs Serializable).
- **Locking:** Optimistic vs. Pessimistic locking strategies.
- **N+1 Query Problem:** Identifying and fixing it using Join Fetching.

6. REST API DESIGN

- **Idempotency:** Designing safe APIs for network retries.
- **Performance:** Pagination, Filtering, and Sorting large datasets.
- **Versioning:** URI vs Header-based versioning strategies.
- **Security:** Implementing JWT Authentication and Rate Limiting.
- **Status Codes:** Using 201, 204, 403, and 409 correctly.

Concepts are 50% of the battle. The other 50% is personalized guidance and mock interviews.